

Shahrim

Shahar muammolarini birgalikda hal qilamiz

Technical Overview

Stack, architecture, data model, security, and how everything is hosted.

Technical Overview

This document describes the technology, architecture, and deployment of Shahrim - a civic issue-reporting platform with one backend serving three clients. It is a pnpm + Turborepo monorepo; the backend is Python/FastAPI; the clients are React and React Native.

1. Architecture at a glance

One backend API is consumed by three frontends:

- **Backend API + Telegram bot** - FastAPI (async) + PostgreSQL/PostGIS.
- **Telegram Mini App** (primary citizen client) - React + TypeScript + Vite.
- **Native mobile app** (iOS + Android) - Expo / React Native.
- **Admin portal** (government operators, desktop web) - React + TypeScript + Vite.

Shared logic lives in workspace packages so the clients stay consistent: the typed API client, the design tokens, and the Uzbek i18n strings.

2. Technology stack

Backend

- **FastAPI** (async) with **SQLAlchemy 2.0** (async ORM) and **Alembic** migrations.
- **PostgreSQL 16 + PostGIS** - geospatial storage for issue locations (Geometry Point, SRID 4326) with a GIST index for maps and district queries.
- **psycopg 3** driver; **pydantic-settings** for env-driven config.
- **aiogram** Telegram bot (runs as its own service).
- **PyJWT** sessions; **bcrypt** for admin passwords.
- AI via the **OpenAI** SDK (GPT-4o vision) behind a pluggable interface.

Clients

- **Mini App / Admin**: React 18 + TypeScript + Vite; i18next; Leaflet + OpenStreetMap for maps; Recharts for admin analytics.
- **Native app**: Expo SDK 51 + expo-router; expo-image-picker, expo-location, react-native-maps, expo-secure-store.

Shared packages

- **@shahrim/api-client** - typed fetch client + domain types.
- **@shahrim/ui-tokens** - Samarkand color/type/spacing tokens.
- **@shahrim/i18n** - Uzbek (Latin) strings; structured for adding more languages.

Tooling

- Docker Compose for local infra; **ruff** (lint+format) for Python; **ESLint/TSC** for JS.
- Tests: **pytest** (backend), **Vitest** (web), **jest-expo** (native). GitHub Actions CI on every push.

3. Repository structure

- **apps/backend** - FastAPI app, bot, Alembic migrations, tests.
- **apps/miniapp** - Telegram Mini App.
- **apps/admin** - admin portal.
- **apps/mobile** - Expo native app.
- **packages/{api-client, ui-tokens, i18n}** - shared code.
- **assets/** - placeholder brand assets + ASSETS.md swap list.

4. Data model

- **User** - telegram_id, name, username, photo, phone, email + password_hash (admins), language, role (citizen/admin).
- **Category** - code, name_uz, icon (9 seeded categories).
- **Issue** - user, photo_url, ai/user/final description, category, urgency, status, lat/lng + PostGIS geom, address/district, timestamps.
- **Resolution** - issue, admin, result_photo_url, note, resolved_at.
- **Rating** - issue (unique), user, stars (1-5), comment.
- **StatusHistory** - issue, status, changed_by, note, timestamp (the timeline).
- **LoginCode** - one-time nonce for native-app deep-link auth.

5. Authentication

- **Mini App**: the client sends Telegram **initData**; the backend validates its HMAC signature with the bot token (never trusts the client), upserts the user, and issues a JWT.
- **Native app**: the app generates a nonce and opens the bot with a deep link; after the user shares their phone, the bot binds the nonce to the user; the app exchanges the nonce for a JWT (one-time).
- **Admin**: email + password (bcrypt) -> JWT; a role check (get_current_admin) enforces RBAC on every admin route.

6. AI pipeline

Photo analysis sits behind an **AIProvider** interface. The default is OpenAI GPT-4o vision with a strict Uzbek JSON-only prompt returning suggestions, category, urgency, and an is_valid_city_issue flag. Guardrails validate and coerce the output, retry once on bad JSON, and fall back safely - an AI failure never blocks a report. When no API key is set, a mock provider keeps the whole flow working in development.

7. Storage, maps, notifications

- **Photos**: a Storage interface with a local-filesystem implementation today (served under /media); an S3-compatible backend can drop in without touching callers.
- **Maps**: Leaflet + OpenStreetMap (no key) behind an abstraction; Yandex can replace it for better Uzbekistan coverage in the pilot.
- **Notifications**: on every status change the backend sends the citizen an Uzbek Telegram message via the Bot API (non-blocking).

8. Hardening

- Per-IP rate limiting on abuse-prone POST endpoints (report, upload, auth) returning HTTP 429.
- Structured request logging (method, path, status, duration).
- Secrets only via environment variables; nothing sensitive is committed.

9. Deployment and hosting

The pieces are hosted differently because a native app is a client, not a website.

Backend (server-hosted)

- Run the same Docker Compose stack (or Kubernetes) on a VPS / cloud host.
- Put a reverse proxy (Caddy, Nginx, or Traefik) in front for a real domain + automatic TLS.
- Use managed PostgreSQL with the PostGIS extension; use an S3-compatible bucket for photos (swap the Storage backend).
- Provide secrets (bot token, JWT secret, OpenAI key, DB URL, admin credentials) as environment variables / a secrets manager.
- The bot runs as its own service; for production, a Telegram webhook can replace long-polling.

Mini App and Admin portal (static web, server-hosted)

- Build each with Vite and serve the static output over HTTPS (any static host / CDN, or behind the same reverse proxy).
- The Mini App must be served from a single fixed HTTPS domain (Telegram is enabling origin-restriction protection for Mini Apps on 20 July 2026).
- The admin portal sits on its own domain/subdomain, behind login.

Native app (built and distributed, not hosted)

The Expo app is not hosted like a web page - it is compiled to native binaries and installed on phones, and it talks to the hosted backend over HTTPS. The standard flow:

- **EAS Build** produces the iOS (.ipa) and Android (.aab/.apk) binaries in the cloud - no local Xcode/Android Studio required to build, though those are needed for local simulators.
- Publish to the **App Store** and **Google Play** (or use internal / enterprise distribution for a closed pilot).
- **EAS Update** pushes over-the-air JavaScript updates between store releases.
- The app reads the backend URL from an environment/config value (EXPO_PUBLIC_API_URL), so pointing it at production is a config change.
- During development the app runs in **Expo Go** on a real phone against a tunnelled backend - no store submission needed.

This is a mainstream, well-supported pattern and works reliably: server-hosted backend + web builds, store-distributed native client, OTA updates for fast iteration.

10. Testing and CI

The suite has 91 automated tests: backend 51 (pytest - auth/initData, issues, AI guardrails, notifications, admin RBAC/analytics/resolve, rating, rate limiting, native exchange), Mini App 17 and admin 7 (Vitest), and native 16 (jest-expo). GitHub Actions runs lint + migrate + tests on each push; a full Metro bundle validates the native app.

11. Open decisions / roadmap

- AI provider + key (OpenAI selected; mock until a key is set).
- Map provider (OSM now; Yandex for pilot coverage).
- Photo storage (local now; S3-compatible for production).
- Government partnership - which office receives and acts on reports.
- Per-district routing and the super-admin management screens.